LAPORAN PRAKTIKUM STRUKTUR DATA

Queue

disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : DR.Wahyudi, S.T, M.T

Asisten Praktikum : Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum pada mata kuliah Struktur Data. Adapun laporan ini membahas mengenai implementasi struktur data *queue* (antrian) menggunakan bahasa pemrograman Java, baik dengan menggunakan array maupun *LinkedList*. Selain itu, laporan ini juga membahas operasi-operasi dasar pada queue seperti *enqueue*, *dequeue*, *peek*, serta penerapan queue dalam beberapa kasus seperti pembalikan data menggunakan *stack* dan iterasi menggunakan *iterator*. Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Padang, 29 April 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat 2
BAB II **3**
PEMBAHASAN **3**
2.1 Pengertian Queue 3
2.2 Operasi dalam Queue 3
2.3 Praktikum 3
2.3.1 Alat dan Bahan 3
2.3.2 Program Class QueueArray 4
2.3.3 Program Class QueueArrayDriver 7
2.3.4 Program Class QueueLinkedList 10
2.3.5 Program Class ReverseData 11
2.3.6 Program Class IterasiQueue 13
BAB III **15**
KESIMPULAN DAN SARAN **15**
3.1 Kesimpulan 15
3.2 Saran 15
DAFTAR PUSTAKA **16**
Penjelasan Tugas **17**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pemahaman yang baik terhadap konsep dasar dalam ilmu komputer, salah satunya adalah struktur data. Struktur data merupakan cara untuk menyimpan, mengelola, dan mengorganisasikan data agar dapat digunakan secara efisien. Dalam mata kuliah Struktur Data, pemahaman terhadap berbagai jenis struktur data sangat penting untuk menunjang kemampuan dalam pemrograman.

Salah satu struktur data dasar yang sering digunakan adalah *queue* (antrian). Queue bekerja dengan prinsip *First In First Out* (FIFO), yaitu data yang pertama kali masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian di loket, sistem antrean pada layanan pelanggan, serta pengelolaan proses dalam sistem komputer.

Pada praktikum ini, dilakukan implementasi struktur data queue menggunakan bahasa pemrograman Java, baik dengan menggunakan array maupun *LinkedList*. Selain itu, juga dipelajari operasi-operasi dasar pada queue seperti penambahan data (*enqueue*), penghapusan data (*dequeue*), serta melihat elemen terdepan dan terakhir. Praktikum ini juga mencakup penerapan queue dalam kasus tertentu seperti pembalikan data dengan bantuan *stack* serta proses iterasi menggunakan *iterator*.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data *queue* (antrian).
2. Mengetahui prinsip kerja *First In First Out* (FIFO).
3. Mampu mengimplementasikan queue menggunakan array dan *LinkedList* dalam bahasa pemrograman.
4. Memahami dan mengaplikasikan operasi dasar queue seperti *enqueue*, *dequeue*, *peek*, dan *size*.

5. Mampu menerapkan queue dalam penyelesaian kasus tertentu, seperti pembalikan data menggunakan *stack*.
6. Memahami cara melakukan iterasi terhadap queue menggunakan *iterator*.

1.3 Manfaat

Praktikum ini memberikan berbagai manfaat bagi mahasiswa dalam memahami konsep struktur data, khususnya *queue*. Melalui kegiatan ini, mahasiswa dapat mengetahui prinsip kerja *First In First Out* (FIFO) serta mampu mengimplementasikan queue menggunakan berbagai pendekatan, seperti array dan *LinkedList* dalam bahasa pemrograman. Selain itu, praktikum ini juga membantu meningkatkan kemampuan logika dan pemecahan masalah, terutama dalam menerapkan operasi-operasi dasar seperti *enqueue*, *dequeue*, *peek*, dan *size*. Dengan adanya latihan secara langsung, mahasiswa memperoleh pengalaman praktis yang dapat menunjang pemahaman teori serta mempersiapkan diri dalam menghadapi permasalahan pemrograman yang lebih kompleks di masa mendatang.

BAB II

PEMBAHASAN

2.1 Pengertian Queue

Queue atau antrian merupakan salah satu struktur data linear yang menerapkan prinsip *First In First Out* (FIFO), yaitu elemen yang pertama kali masuk akan menjadi elemen pertama yang keluar. Konsep queue serupa dengan antrian dalam kehidupan sehari-hari, seperti antrian di kasir atau layanan publik, di mana orang yang datang lebih dahulu akan dilayani terlebih dahulu.

Dalam implementasinya, queue memiliki dua ujung utama, yaitu front (depan) sebagai tempat pengambilan data dan rear (belakang) sebagai tempat penambahan data. Queue dapat diimplementasikan menggunakan berbagai cara, seperti array maupun *LinkedList*, tergantung pada kebutuhan dan efisiensi yang diinginkan.

2.2 Operasi dalam Queue

a) Enqueue → operasi untuk menambahkan elemen ke dalam queue. Penambahan dilakukan di bagian belakang (rear) antrian.
b) Dequeue → operasi untuk menghapus atau mengambil elemen dari queue. Penghapusan dilakukan pada bagian depan (front) antrian sesuai prinsip FIFO
c) Peek/Front → operasi untuk melihat elemen yang berada di posisi depan tanpa menghapusnya dari queue
d) Rear → operasi untuk melihat elemen yang berada pada posisi paling belakang
e) Size → operasi untuk mengetahui jumlah elemen yang terdapat dalam queue.

2.3 Praktikum

2.3.1 Alat dan Bahan

1. Laptop/computer dengan eclipse terinstal.
2. Materi praktikum mengenai queue

2.3.2 Program Class QueueArray

```java
package pekan4_2511532019;

public class QueueArray_2511532019 {
    int front_2019, rear_2019, size_2019;
    int capacity_2019;
    int array_2019[];

    public QueueArray_2511532019(int capacity_2019) {
            this.capacity_2019=capacity_2019;
            front_2019=this.size_2019=0;
            rear_2019=capacity_2019-1;
            array_2019=new int [this.capacity_2019];
    }

    boolean isFull_2019 (QueueArray_2511532019 queue_2019) {
            return  (queue_2019.size_2019 == queue_2019.capacity_2019);
    }

    boolean isEmpty_2019 (QueueArray_2511532019 queue_2019) {
            return (queue_2019.size_2019==0);
    }

    void enqueue_2019 (int item_2019) {
            if (isFull_2019(this))
                    return;
            this.rear_2019=(this.rear_2019+1)%this.capacity_2019;
            this.array_2019 [this.rear_2019]=item_2019;
            this.size_2019=this.size_2019+1;
            System.out.println(item_2019 + " enqueued to queue");
    }
    int dequeue_2019() {
            if (isEmpty_2019(this))
                    return Integer.MIN_VALUE;
            int item_2019=this.array_2019[this.front_2019];
            this.front_2019=(this.front_2019+1)%this.capacity_2019;
            this.size_2019=this.size_2019-1;
            return item_2019;
    }
    int front_2019() {
            if(isEmpty_2019(this))
                    return Integer.MIN_VALUE;
            return this.array_2019[this.front_2019];
    }
    int rear_2019() {
            if (isEmpty_2019(this))
                    return Integer.MIN_VALUE;
            return this.array_2019[this.rear_2019];
    }
    //mencetak elemen antrian
    void display_2019() {
            int i;
            if(front_2019 == rear_2019) {
                    System.out.printf("\nAntrian Kosong\n");
                    return;
            }
```

```
        //kunjungi dari belakang dan cetaak
        for (i=front_2019;i<rear_2019;i++) {
            System.out.printf("%d <-- ",array_2019[i]);
        }
        return;
    }
}
```

1. `package pekan4_2511532019;`
   kode berada dalam peckage Bernama pekan4_2511532019
2. `public class QueueArray_2511532019 {`
   membuat class Bernama QueryArray_2511532019
3. `int front_2019, rear_2019, size_2019;`
   front menunjuk elemen paling depan (yang akan keluar), rear menunjuk elemen paling belakan (yang terakhir masuk), dan size menunjuk jumlah elemen saat ini
4. `int capacity_2019;`
   kapasitas maksimal queue
5. `int array_2019[];`
   tempat menyimpan data queue
6. `public QueueArray_2511532019(int capacity_2019) {`
   constructor class, dipanggil saat objek dibuat
7. `this.capacity_2019=capacity_2019;`
   mengisi kapasitas queue sesuai input
8. `front_2019=this.size_2019=0;`
   menginisialisasi front = 0, size = 0
9. `rear_2019=capacity_2019-1;`
   set posisi awal rear di belakang agar enqueue pertama bekerja benar
10. `array_2019=new int [this.capacity_2019];`
    membuat array dengan ukuran sesuai kapasitas
11. `boolean isFull_2019 (QueueArray_2511532019 queue_2019) {`
    method untuk mengecek apakan queue penuh
12. `return  (queue_2019.size_2019 == queue_2019.capacity_2019);`
    mengembalikan nilai true jika jumlah elemen = kapsitas
13. `boolean isEmpty_2019 (QueueArray_2511532019 queue_2019) {`
    method untuk mengecek apakah queue kosong

14. `return (queue_2019.size_2019==0);`

    mengembalikan true jika queue kosong

15. `void enqueue_2019 (int item_2019) {`

    method untuk menambahkan data ke queue

16. `if (isFull_2019(this))`

    mengecek apakah queue penuh

17. `return;`

    jika penuh, keluar langsung dari method

18. `this.rear_2019=(this.rear_2019+1)%this.capacity_2019;`

    menggeser posisi rear ke depan

19. `this.array_2019 [this.rear_2019]=item_2019;`

    menyimpan data ke posisi rear

20. `this.size_2019=this.size_2019+1;`

    menambah jumlah elemen

21. `System.out.println(item_2019 + " enqueued to queue");`

    Menampilkan data yang berhasil ditambahkan

22. `int dequeue_2019() {`

    method untuk mengambil data dari queue

23. `if (isEmpty_2019(this))`

    mengecek apakah queue kosong

24. `return Integer.MIN_VALUE;`

    jiak kosong, mengembalikan nilai minimum

25. `int item_2019=this.array_2019[this.front_2019];`

    mengambil data dari posisi front

26. `this.front_2019=(this.front_2019+1)%this.capacity_2019;`

    menggeser front ke depan

27. `this.size_2019=this.size_2019-1;`

    mengurangi jumlah elemen

28. `return item_2019;`

    mengembalikan nilai yang diambil

29. `int front_2019() {`

    method untuk melihat elemen depan

30. `if(isEmpty_2019(this))`

cek apakah kosong

31. `return Integer.MIN_VALUE;`

    jika kosong, return nilai minimum

32. `return this.array_2019[this.front_2019];`

    mengembalikan nilai elemen depan

33. `int rear_2019() {`

    method untuk melihat elemen belakang

34. `if (isEmpty_2019(this))`

    cek apakah kosong

35. `return Integer.MIN_VALUE;`

    jika kosong, return nilai minimum

36. `return this.array_2019[this.rear_2019];`

    mengembalikan nilai elemen belakang

37. `void display_2019() {`

    method untuk menampilkan isi queue

38. `int i;`

    deklarasi variabel i

39. `if(front_2019 == rear_2019) {`

    mengecek kondisi kosong

40. `System.out.printf("\nAntrian Kosong\n");`

    Menampilkan pesan queue kosong

41. `return;`

    keluar dari method

42. `for (i=front_2019;i<rear_2019;i++) {`

    loop dari depan ke belakang

43. `System.out.printf("%d <-- ",array_2019[i]);`

    Menampilkan setiap elemen queue

44. `return;`

    keluar dari method

2.3.3 Program Class QueueArrayDriver

```
package pekan4_2511532019;

public class QueueArrayDriver_2511532019 {
```

```
    public static void main(String[] args) {
            QueueArray_2511532019 queue_2019=new
QueueArray_2511532019(1000);
            queue_2019.enqueue_2019(10);
            queue_2019.enqueue_2019(20);
            queue_2019.enqueue_2019(30);
            queue_2019.enqueue_2019(40);
            System.out.println("Item di depan "+
queue_2019.front_2019());
            System.out.println("Item paling belakang
"+queue_2019.rear_2019());
            System.out.println("tampilan queue");
            queue_2019.display_2019();
            System.out.println();
            System.out.println(queue_2019.dequeue_2019()+ " dihapus
dari queue");
            System.out.println("item di depan: "+
queue_2019.front_2019());
            System.out.println("item belakan : " +
queue_2019.rear_2019());
            System.out.println("tampilan queue setelah satu data
dihapus");
            queue_2019.display_2019();
    }
}
```

1. `package pekan4_2511532019;`

   menunjukkan class ini berada pada package 4

2. `public class QueueArrayDriver_2511532019 {`

   mendeklarasikan class utama untuk menjalankan program queue

3. `public static void main(String[] args) {`

   method main adalah titik awal eksekusi program

4. `QueueArray_2511532019 queue_2019=new QueueArray_2511532019(1000);`

   Membuat objek queue Bernama queue_2019

5. `queue_2019.enqueue_2019(10);`

   menambahkan angka 10 ke dalam queue

6. `queue_2019.enqueue_2019(20);`

   menambahkan angka 20 ke dalam queue

7. `queue_2019.enqueue_2019(30);`

   menambahkan angka 30 ke dalam queue

8. `queue_2019.enqueue_2019(40);`

   menambahkan angka 40 ke dalam queue

9. `System.out.println("Item di depan "+ queue_2019.front_2019());`

Menampilkan elemen paling depan, output = 10

10. ```
System.out.println("Item paling belakang "
+queue_2019.rear_2019());
```

    Menampilkan elemen paling belakang, output=40

11. ```
System.out.println("tampilan queue");
```

    Menampilkan teks “tampilan queue”

12. ```
queue_2019.display_2019();
```

    memanggil method untuk menampilan isi queue

13. ```
System.out.println();
```

    Baris kosong

14. ```
System.out.println(queue_2019.dequeue_2019()+ " dihapus dari
queue");
```

    Menghapus elemen paling depan, elemen yang dihapus = 10, menampilan pesan “10 dihapus dari queue”

15. ```
System.out.println("item di depan: "+ queue_2019.front_2019());
```

    Menampilkan elemen depan yang baru, output = 20

16. ```
System.out.println("item belakan : " + queue_2019.rear_2019());
```

    Menampilkan elemen belakang, output = 40

17. ```
System.out.println("tampilan queue setelah satu data dihapus");
```

    Menampilkan pesan “tampilan queue setelah satu data dihapus”

18. ```
queue_2019.display_2019();
```

    menampilkan isi queue setelah penghapusan

**Output**

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
item di depan: 20
item belakan : 40
tampilan queue setelah satu data dihapus
20 <-- 30 <--
```
```

2.3.4 Program Class QueueLinkedList

```
package pekan4_2511532019;

import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511532019 {

    public static void main(String[] args) {
            Queue<Integer>q_2019=new LinkedList<>();
            //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
            for(int i=0; i<6; i++)
                    q_2019.add(i);
            //menampilkan isi antrian
            System.out.println("Elemen Antrian"+q_2019);
            //untuk menghapus kepala antrian
            int hapus_2019=q_2019.remove();
            System.out.println("Hapus elemen = "+hapus_2019);
            System.out.println(q_2019);
            //untuk melihat antrian terdepan
            int depan_2019=q_2019.peek();
            System.out.println("Kepala antrian = "+depan_2019);
            int banyak_2019=q_2019.size();
            System.out.println("Size Antrian = "+banyak_2019);
    }
}
```

1. `package pekan4_2511532019;`

   menentukan peckage tempat class ini disimpan

2. `import java.util.Queue;`

   mengimpor interface queue dari library java

3. `import java.util.LinkedList;`

   mengimpor class LinkedList

4. `public class QueueLinkedList_2511532019 {`

   mendeklarasikan class

5. `public static void main(String[] args) {`

   method utama, titik awal eksekusi program

6. `Queue<Integer>q_2019=new LinkedList<>();`

   Membuat objek queue Bernama q_2019, tipe data Integer, implementasi menggunakan LinkedList

7. `for(int i=0; i<6; i++)`

   perulangan dari 0 sampai 5

8. `q_2019.add(i);`

menambahkan elemen ke queue

9. `System.out.println("Elemen Antrian"+q_2019);`

   Menampilkan seluruh isi Queue

10. `int hapus_2019=q_2019.remove();`

    mengahapus elemen paling depan, yang dihapus = 0

11. `System.out.println("Hapus elemen = "+hapus_2019);`

    Menampilkan elemen yang dihapus

12. `System.out.println(q_2019);`

    Menampilkan queue setelah penghapusan

13. `int depan_2019=q_2019.peek();`

    mengambil elemen depan tanpa meghapusnya

14. `System.out.println("Kepala antrian = "+depan_2019);`

    Menampilkan elemen depan

15. `int banyak_2019=q_2019.size();`

    mengambil jumlah elemen dalam queue

16. `System.out.println("Size Antrian = "+banyak_2019);`

    Menampilkan jumlah elemen

Output

```
Elemen Antrian[0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala antrian = 1
Size Antrian = 5
```

2.3.5 Program Class ReverseData

```
package pekan4_2511532019;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532019 {

    public static void main(String[] args) {
            Queue<Integer>q_2019=new LinkedList<Integer>();
            q_2019.add(1);
            q_2019.add(2);
            q_2019.add(3);//[1,2,3]
            System.out.println("Sebelum reverse = "+q_2019);
            Stack<Integer>s_2019=new Stack<Integer>();
            while (!q_2019.isEmpty()) {//Q -> S
```

```
                s_2019.push(q_2019.remove());
            }
            while (!s_2019.isEmpty()) { //S -> Q
                q_2019.add(s_2019.pop());
            }
            System.out.println("Sesudah reverse = "+q_2019);
    }
}
```

1. `package pekan4_2511532019;`

   menentukan peckage tempat class ini disimpan

2. `import java.util.LinkedList;`

   mengimpor class LinkedList untuk implrmrntasi queue

3. `import java.util.Queue;`

   mengimpor interface queue

4. `import java.util.Stack;`

   mengimpor class stack

5. `public class ReverseData_2511532019 {`

   mendeklarasikan class

6. `public static void main(String[] args) {`

   method utama, titik awal program

7. `Queue<Integer>q_2019=new LinkedList<Integer>();`

   Membuat queue bertipe Integer menggunakan LinkedList

8. `q_2019.add(1);`

   menambahkan angka sat uke queue

9. `q_2019.add(2);`

   menambahkan angka 2 ke queue

10. `q_2019.add(3);`

    menambahkan angka 3 ke queue, isi queue sekarang [1,2,3]

11. `System.out.println("Sebelum reverse = "+q_2019);`

    Menampilkan isi queue sebelum dibalik

12. `Stack<Integer>s_2019=new Stack<Integer>();`

    Membuat stack kosong

13. `while (!q_2019.isEmpty()) {`

    perulangan terjadi Selama queue tidak kosong

14. `s_2019.push(q_2019.remove());`

ambil elemen dari queue, masukkan ke stack

15. `while (!s_2019.isEmpty()) {`

    perulangan selama tidak kosong

16. `q_2019.add(s_2019.pop());`

    ambil dari stack, masukkan Kembali ke queue

17. `System.out.println("Sesudah reverse = "+q_2019);`

    Menampilkan hasil setelah dibalik

Output

```
Sebelum reverse = [1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

2.3.6 Program Class IterasiQueue

```java
package pekan4_2511532019;

import java.util.Queue;
import java.util.Iterator;
import java.util.LinkedList;

public class IterasiQueue_2511532019 {

    public static void main(String[] args) {
            Queue<String>q_2019=new LinkedList<>();
            q_2019.add("Praktikum");
            q_2019.add("Struktur");
            q_2019.add("Data");
            q_2019.add("Dan");
            q_2019.add("Algoritma");
            Iterator<String>iterator_2019=q_2019.iterator();
            while (iterator_2019.hasNext()) {
                    System.out.print(iterator_2019.next()+ " ");
            }
    }
}
```

1. `package pekan4_2511532019;`

   menentukan peckage tempat class ini disimpan

2. `import java.util.Queue;`

   mengimpor interface queue

3. `import java.util.Iterator;`

   mengimpor iterator untuk menelusuri elemen satu per satu

4. `import java.util.LinkedList;`

   mengimpor LinkedList sebagai implementasi queue

5. `public class IterasiQueue_2511532019 {`

   mendeklarasikan class

6. `public static void main(String[] args) {`

   method utama, titik awal program

7. `Queue<String>q_2019=new LinkedList<>();`

   Membuat queue bertipe string menggunakan LinkedList

8. `q_2019.add("Praktikum");`

   menambahkan elemen “Praktikum” ke queue

9. `q_2019.add("Struktur");`

   menambahkan elemen “Struktur” ke queue

10. `q_2019.add("Data");`

    menambahkan elemen “Data” ke queue

11. `q_2019.add("Dan");`

    menambahkan elemen “dan” ke queue

12. `q_2019.add("Algoritma");`

    menambahkan elemen “Algoritma” ke queue

13. `Iterator<String>iterator_2019=q_2019.iterator();`

    Membuat objek iterator untuk menelusuri queue, iterator akan mulai dari elemen paling depan

14. `while (iterator_2019.hasNext()) {`

    perulangan selama masih ada elemen berikutnya

15. `System.out.print(iterator_2019.next()+ " ");`

    Mengambil elemen berikutnya lalu mencetaknya dengan menambahkan spasi sekaligus memindahkan posisi iterator

Output

```
Praktikum Struktur Data Dan Algoritma
```

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *queue* merupakan salah satu struktur data yang bekerja berdasarkan prinsip *First In First Out* (FIFO), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Implementasi queue dapat dilakukan dengan berbagai cara, seperti menggunakan array maupun *LinkedList*, yang masing-masing memiliki kelebihan dan kekurangan. Dalam praktikum ini, berbagai operasi dasar queue seperti *enqueue*, *dequeue*, *peek*, dan *size* telah berhasil diimplementasikan dan dijalankan dengan baik. Selain itu, pemanfaatan queue dalam kasus tertentu seperti pembalikan data dengan bantuan *stack* serta proses iterasi menggunakan *iterator* juga menunjukkan bahwa queue memiliki peran penting dalam pengolahan data secara terstruktur. Dengan demikian, praktikum ini membantu meningkatkan pemahaman konsep sekaligus keterampilan pemrograman dalam mengelola struktur data queue.

3.2 Saran

1. lebih banyak berlatih dalam mengimplementasikan queue agar semakin memahami konsepnya
2. Perlu meningkatkan ketelitian dalam penulisan kode untuk menghindari kesalahan (*error*) saat program dijalankan.

DAFTAR PUSTAKA

[1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ: Wiley, 2014.

Penjelasan Tugas

Program Class AntrianLoket

```
package pekan4_2511532019;

import java.util.*;

public class AntrianLoket_2511532019 {

	String queue_2019[];
	int front_2019, rear_2019, max_2019;

	public AntrianLoket_2511532019(int max_2019) {
		this.max_2019 = max_2019;
		queue_2019 = new String[max_2019];
		front_2019 = -1;
		rear_2019 = -1;
	}

	boolean isEmpty_2019() {
		return (front_2019 == -1);
	}

	boolean isFull_2019() {
		return (rear_2019 == max_2019 - 1);
	}

	void enqueue_2019(String data) {
		if (isFull_2019()) {
			System.out.println("Antrian penuh!");
		} else {
			if (isEmpty_2019()) {
				front_2019 = 0;
			}
			rear_2019++;
			queue_2019[rear_2019] = data;
			System.out.println("Data berhasil ditambahkan ke
antrian");
		}
	}

	void dequeue_2019() {
		if (isEmpty_2019()) {
			System.out.println("Antrian kosong!");
		} else {
			System.out.println(queue_2019[front_2019] + " telah
dilayani");
			front_2019++;
			if (front_2019 > rear_2019) {
				front_2019 = rear_2019 = -1;
			}
		}
	}
```

```
        void display_2019() {
                if (isEmpty_2019()) {
                        System.out.println("Antrian kosong");
                } else {
                        System.out.println("Isi antrian:");
                        int no = 1;
                        for (int i = front_2019; i <= rear_2019; i++) {
                                System.out.println(no + ". " +
queue_2019[i]);
                                no++;
                        }
                }
        }

        void reverse_2019() {
                if (isEmpty_2019()) {
                        System.out.println("Antrian kosong");
                } else {
                        Stack<String> stack_2019 = new Stack<>();
                        for (int i = front_2019; i <= rear_2019; i++) {
                                stack_2019.push(queue_2019[i]);
                        }
                        for (int i = front_2019; i <= rear_2019; i++) {
                                queue_2019[i] = stack_2019.pop();
                        }
                        System.out.println("Antrian berhasil dibalik");
                        display_2019();
            }
        }

}
```

1. `package pekan4_2511532019;`

   menentukan peckage tempat class disimpan

2. `import java.util.*;`

   mengimpor semua class yang ada di package java

3. `public class AntrianLoket_2511532019 {`

   mendeklarasikan class

4. `String queue_2019[];`

   Array bertipe string untuk menyimpan data pelanggan dalam antrian

5. `int front_2019, rear_2019, max_2019;`

   deklarasi variabel

6. `public AntrianLoket_2511532019(int max_2019) {`

   constructor untuk menginisialisasi queue saat objek dibuat

7. `this.max_2019 = max_2019;`

   menyimpan nilai kapasitas ke variabel max

8. `queue_2019 = new String[max_2019];`

   membuat array dengan ukuran sesuai kapasitas

9. `front_2019 = -1;`

   menandakan queue masih kosong

10. `rear_2019 = -1;`

    posisi belakang juga kosong

11. `boolean isEmpty_2019() {`

    method untuk mengecek apakah queue kosong

12. `return (front_2019 == -1);`

    jika front = -1, berarti queue kosong, return true

13. `boolean isFull_2019() {`

    method untuk mengecek apakah queue penuh

14. `return (rear_2019 == max_2019 - 1);`

    jika rear sudah di posisi terakhir array berarti queue penuh

15. `void enqueue_2019(String data) {`

    method untuk menambahkan data ke queue

16. `if (isFull_2019()) {`

    mengecek apakah queue penuh

17. `System.out.println("Antrian penuh!");`

    Jika penuh, tampilkan pesan “antrian penuh”

18. `} else {`

    Jika tidak penuh, lanjut tambah data

19. `if (isEmpty_2019()) {`

    mengecek apakah queue kosong

20. `front_2019 = 0;`

    jika kosong, set posisi depan ke indeks 0

21. `rear_2019++;`

    geser posisi rear ke belakang

22. `queue_2019[rear_2019] = data;`

    menyimpan data ke array

23. `System.out.println("Data berhasil ditambahkan ke antrian");`

    Menampilkan pesan sukses ditambahkan

24. `void dequeue_2019() {`

method untuk meghapus data dari queue

25. `if (isEmpty_2019()) {`

    mengecek apakah queue kosong

26. `System.out.println("Antrian kosong!");`

    Jika kosong tampilkan pesan antrian kosong

27. `} else {`

    Kalua tidak kosong, lanjut hapus data

28. `System.out.println(queue_2019[front_2019] + " telah dilayani");`

    Menampilkan data yang dihapus

29. `front_2019++;`

    geser posisi front ke depan

30. `if (front_2019 > rear_2019) {`

    jika semua data sudah habis

31. `front_2019 = rear_2019 = -1;`

    reset queue ke kondisi kosong

32. `void display_2019() {`

    method untuk menampilkan isi queue

33. `if (isEmpty_2019()) {`

    cek apakan queue kosong

34. `System.out.println("Antrian kosong");`

    Jika kosong tampilkan pesan antrian kosong

35. `} else {`

    `System.out.println("Isi antrian:");`

    Jika ada data, tampilkan isi

36. `int no = 1;`

    variabel untuk nomor urut

37. `for (int i = front_2019; i <= rear_2019; i++) {`

    loop dari depan ke belakang

38. `System.out.println(no + ". " + queue_2019[i]);`

    Menampilkan data dengan nomor

39. `no++;`

    menambah nomor urut

40. `void reverse_2019() {`

method untuk membalik urutan antrian

41. `if (isEmpty_2019()) {`

    mengecek apakah queue kosong

42. `System.out.println("Antrian kosong");`

    Jika kosong tampilkan pesan antrian kosong

43. `} else {`

    Jika tidak kosong

44. `Stack<String> stack_2019 = new Stack<>();`

    Buat stack kosong untuk proses pembalikan

45. `for (int i = front_2019; i <= rear_2019; i++) {`

    perulangan dari elemen depan hingga belakang queue

46. `stack_2019.push(queue_2019[i]);`

    mengambil setiap elemen dari queue dan memasukkannya ke stack

47. `for (int i = front_2019; i <= rear_2019; i++) {`

    perulangan Kembali dari depan ke belakang

48. `queue_2019[i] = stack_2019.pop();`

    mengambil elemen dari stack dan memasukkannya Kembali ke queue

49. `System.out.println("Antrian berhasil dibalik");`

    Menampilkan pesan berhasil

50. `display_2019();`

    menampilkan isi queue setelah dibalik

Program Class AntrianLoketDriver

```
package pekan4_2511532019;

import java.util.Scanner;

public class AntrianLoketDriver_2511532019 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        AntrianLoket_2511532019 q = new AntrianLoket_2511532019(5);
        int pilihan;
        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
```

```java
            System.out.print("Pilih menu: ");
            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = input.nextLine();
                    q.enqueue_2019(nama);
                    break;
                case 2:
                    q.dequeue_2019();
                    break;
                case 3:
                    q.display_2019();
                    break;
                case 4:
                    q.reverse_2019();
                    q.display_2019();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid");
            }

        }
        while (pilihan != 5);
        input.close();
    }
}
```

1. `package pekan4_2511532019;`

   menentukan package tempat class disimpan

2. `import java.util.Scanner;`

   mengimpor class scanner untuk menerima input dari pengguna

3. `public class AntrianLoketDriver_2511532019 {`

   mendeklarasikan class driver

4. `public static void main(String[] args) {`

   method utama, program mulai dijalankan

5. `Scanner input = new Scanner(System.in);`

   Membuat objek scanner untuk membaca input dari keyboard

6. `AntrianLoket_2511532019 q = new AntrianLoket_2511532019(5);`

   Membuat objek queue dengan kapasitas maksimal 5 elemen

7. `int pilihan;`

variabel untuk menyimpan pilihan menu dari user

8. `do {`

   memulai peerulangan menu, akan terus dilakukan sampai user memilih keluar

9. `System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");`

   Menampilkan judul program

10. `System.out.println("1. Tambah Antrian");`

    Menampilkan menu tambah data

11. `System.out.println("2. Hapus Antrian");`

    Menampilkan menu hapus data

12. `System.out.println("3. Tampilkan Antrian");`

    Menampilkan menu untuk melihat isi queue

13. `System.out.println("4. Reverse");`

    Menampilkan menu untuk membalik antrian

14. `System.out.println("5. Keluar");`

    Menampilkan menu keluar

15. `System.out.print("Pilih menu: ");`

    Meminta user memilih menu

16. `pilihan = input.nextInt();`

    membaca input angka dari user

17. `input.nextLine();`

    enter

18. `switch (pilihan) {`

    digunakan untuk menentukan aksi berdasarkan pilihan user

19. `case 1:`

    jika user memilih menu 1

20. `System.out.print("Masukkan nama pelanggan: ");`

    Meminta input nama pelanggan

21. `String nama = input.nextLine();`

    Menyimpan nama pelanggan

22. `q.enqueue_2019(nama);`

    menambahkan data ke queue

23. `break;`

mengakhiri case 1

24. `case 2:`

    jika user memilih menu 2

25. `q.dequeue_2019();`

    menghapus data dari depan queue

26. `break;`

    mengakhiri case 2

27. `case 3:`

    jika user memilih menu 3

28. `q.display_2019();`

    menampilkan isi queue

29. `break;`

    mengakhiri case 3

30. `case 4:`

    jika user memilih menu 4

31. `q.reverse_2019();`

    membalik urutan queue

32. `q.display_2019();`

    langsung menampilkan hasil setelah dibalik

33. `break;`

    mengakhiri case 4

34. `case 5:`

    jika user memilih menu 5

35. `System.out.println("Program selesai");`

    Menampilkan pesan selesai

36. `break;`

    mengakhiri case 5

37. `default:`

    jika input tidak sesuai menu

38. `System.out.println("Pilihan tidak valid");`

    Menampilkan pesan pilihan tidak valid

39. `while (pilihan != 5);`

    program akan terus berjalan selama user tidak memilih 5

40. `input.close();`

menutup objek scanner

Output

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Siti
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi
3. Siti

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. Siti
2. Budi
3. Andi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Siti telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Andi telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```